# Academy Hub — Sesión 8

## Django REST Framework: API REST, serializers, ViewSets, routers, filtros, permisos y documentación

> **Antes de empezar:** la Sesión 7 debe estar cerrada con todos los tests en verde. Si alguno de los tests de constraint de `Convocatoria` sigue fallando, aplica `with transaction.atomic():` **dentro** de `assertRaises(IntegrityError)` en los cuatro tests de `Convocatoria`.

---

## Índice


1. Objetivo general
2. Qué vamos a aprender
3. Teoría: REST, HTTP, JSON, códigos de estado
4. Instalación de dependencias
5. Configuración en `settings/base.py`
6. Configuración en `config/urls.py`
7. Estructura de la app `api`
8. Permisos personalizados
9. Autenticación por API
10. Serializers por app
11. Filtros por app
12. ViewSets por app
13. Acciones personalizadas
14. Routers
15. Documentación OpenAPI + Swagger + ReDoc
16. Tests por app
17. Ejemplos con curl
18. Checklist global
19. Registro de sesión
20. Commit recomendado


---

## 1. Objetivo general

Hasta ahora Academy Hub es una aplicación web Django:

```
Navegador → URL → View → Template → HTML
```

Ahora añadimos una segunda vía de acceso:

```
Cliente → API REST → DRF → ORM → PostgreSQL
```

React + Vite (o cualquier frontend separado, o una app móvil, o scripts) podrá consumir todos los datos que hoy se ven en los templates, **excepto los que solo tienen sentido en el admin de Django** (contenido de la home, páginas informativas).

**Regla de oro:** web y API comparten la misma lógica de negocio. Si una regla existe en la web, existe en la API. Si una regla existe en la API, existe en la web. Se implementa **una vez** en servicios y se llama desde ambas.

---

# 2. Qué vamos a aprender

- REST (recursos, métodos HTTP, códigos de estado).
- JSON vs HTML.
- `Serializer` y `ModelSerializer`.
- Nested serializers.
- Validaciones de campo y globales.
- `APIView`, `Generic Views`, `ViewSet`, `ModelViewSet`.
- `Router` con `DefaultRouter`.
- Permisos: `IsAuthenticatedOrReadOnly`, permisos personalizados de objeto.
- `perform_create()` para asignar propietario.
- Filtros: `django-filter`, `filterset_fields`, `FilterSet`.
- Búsqueda: `SearchFilter`.
- Ordenación: `OrderingFilter`.
- Paginación: `PageNumberPagination`.
- Acciones personalizadas con `@action`.
- Autenticación por sesión (login/logout/me).
- CORS para frontends separados.
- OpenAPI + Swagger + ReDoc.
- Tests con `APITestCase` y `force_authenticate`.

---

# 3. Teoría: REST, HTTP, JSON, códigos de estado

## REST

REST es un estilo de diseño para APIs. Trabajamos con **recursos**:

- `/api/v1/courses/` → colección.
- `/api/v1/courses/15/` → elemento.

## Métodos HTTP

- `GET` → obtener.
- `POST` → crear.
- `PUT` → sustituir.
- `PATCH` → modificar parcialmente.
- `DELETE` → eliminar.

## Códigos de estado

- `200 OK`.
- `201 Created`.
- `204 No Content` (tras DELETE).
- `400 Bad Request` (datos inválidos).
- `401 Unauthorized` (falta autenticación).
- `403 Forbidden` (autenticado pero sin permiso).
- `404 Not Found`.
- `405 Method Not Allowed`.

## JSON

```
{
  "id": 1,
  "title": "Inglés B2 intensivo",
  "price": "480.00"
}
```

---

# 4. Instalación de dependencias

```
pip install djangorestframework
pip install django-filter
pip install drf-spectacular
pip install django-cors-headers
pip freeze > requirements.txt
```

# 5. Configuración en `settings/base.py`

**Archivo:** `config/settings/base.py`.

**Qué hacer:** añadir al final del archivo las nuevas apps y la configuración de DRF, Spectacular y CORS.

## 5.1. Apps de terceros

Busca:

```
THIRD_PARTY_APPS = []
```

Y sustitúyelo por:

```
THIRD_PARTY_APPS = [
    "rest_framework",
    "django_filters",
    "drf_spectacular",
    "corsheaders",
]
```

## 5.2. Apps locales

Busca:

```
LOCAL_APPS = [
    "core",
    "accounts",
    "academies",
    "courses",
    "enrollments",
    "reviews",
    "payments",
    "notifications",
    "evaluations",
    "talent",
    "level_tests",
    "search",
    "dashboard",
]
```

Añade `"api"` al final:

```
LOCAL_APPS = [
    "core",
    "accounts",
    "academies",
    "courses",
    "enrollments",
    "reviews",
    "payments",
    "notifications",
    "evaluations",
    "talent",
    "level_tests",
    "search",
    "dashboard",
    "api",
]
```

## 5.3. Middleware de CORS

Busca `MIDDLEWARE` y añade `"corsheaders.middleware.CorsMiddleware"` **al principio**, antes de `CommonMiddleware`:

```
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

## 5.4. Configuración de DRF, Spectacular y CORS

Añade al final de `base.py`:

```
# ============================================================
# DJANGO REST FRAMEWORK
# ============================================================

REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",
    ],
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.SessionAuthentication",
```

```
    ],
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 12,
    "DEFAULT_FILTER_BACKENDS": [
        "django_filters.rest_framework.DjangoFilterBackend",
        "rest_framework.filters.SearchFilter",
        "rest_framework.filters.OrderingFilter",
    ],
    "DEFAULT_SCHEMA_CLASS": "drf_spectacular.openapi.AutoSchema",
}


# ============================================================
# DRF SPECTACULAR (OpenAPI / Swagger / ReDoc)
# ============================================================

SPECTACULAR_SETTINGS = {
    "TITLE": "Academy Hub API",
    "DESCRIPTION": "API REST de Academy Hub: cursos, academias, inscripciones, pagos y talento.",
    "VERSION": "1.0.0",
    "SERVE_INCLUDE_SCHEMA": False,
    "COMPONENT_SPLIT_REQUEST": True,
}


# ============================================================
# CORS (para React + Vite u otros frontends)
# ============================================================

CORS_ALLOWED_ORIGINS = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
    "http://localhost:3000",
    "http://127.0.0.1:3000",
]

CORS_ALLOW_CREDENTIALS = True

CSRF_TRUSTED_ORIGINS = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
    "http://localhost:3000",
    "http://127.0.0.1:3000",
]
```

## 6. Configuración en `config/urls.py`

**Archivo:** `config/urls.py`.

Añade los `include` y las vistas de Spectacular:

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import include, path

from drf_spectacular.views import (
    SpectacularAPIView,
    SpectacularRedocView,
    SpectacularSwaggerView,
)

urlpatterns = [
    path("admin/", admin.site.urls),

    path("", include("core.urls")),
    path("accounts/", include("accounts.urls")),
    path("academies/", include("academies.urls")),
    path("courses/", include("courses.urls")),
    path("enrollments/", include("enrollments.urls")),
    path("reviews/", include("reviews.urls")),
    path("payments/", include("payments.urls")),
    path("notifications/", include("notifications.urls")),
    path("evaluations/", include("evaluations.urls")),
    path("talent/", include("talent.urls")),
    path("level-tests/", include("level_tests.urls")),
    path("search/", include("search.urls")),
    path("dashboard/", include("dashboard.urls")),

    # API REST v1
    path("api/v1/", include("api.urls")),

    # Documentación OpenAPI
    path("api/schema/", SpectacularAPIView.as_view(), name="schema"),
    path(
        "api/docs/",
        SpectacularSwaggerView.as_view(url_name="schema"),
        name="swagger-ui",
    ),
    path(
        "api/redoc/",
        SpectacularRedocView.as_view(url_name="schema"),
        name="redoc",
    ),
]
```

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

---

# 7. Estructura de la app `api`

Crea la app:

```
python manage.py startapp api
```

Estructura de archivos a crear dentro de `api/`:

```
api/
    __init__.py
    apps.py
    permissions.py
    urls.py
    serializers/
        __init__.py
        accounts.py
        academies.py
        courses.py
        enrollments.py
        reviews.py
        payments.py
        notifications.py
        evaluations.py
        talent.py
        level_tests.py
    filters/
        __init__.py
        courses.py
        academies.py
        enrollments.py
        payments.py
    views/
        __init__.py
        auth.py
        accounts.py
        academies.py
        courses.py
        enrollments.py
```

```
        reviews.py
        payments.py
        notifications.py
        evaluations.py
        talent.py
        level_tests.py
        stats.py
    tests/
        __init__.py
        test_auth.py
        test_courses.py
        test_academies.py
        test_enrollments.py
        test_reviews.py
        test_favorites.py
        test_talent.py
        test_stats.py
```

**Nota:** las carpetas `serializers/`, `filters/`, `views/` y `tests/` necesitan `__init__.py` vacío dentro.

---

# 8. Permisos personalizados

### Archivo: `api/permissions.py`

```python
from rest_framework.permissions import SAFE_METHODS, BasePermission


class IsAcademyOwnerOrReadOnly(BasePermission):
    """Solo el dueño de la academia puede modificar."""

    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.owner == request.user


class IsCourseAcademyOwnerOrReadOnly(BasePermission):
    """Solo el dueño de la academia del curso puede modificar."""

    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.academy.owner == request.user
```

```python
class IsTeacherAcademyOwnerOrReadOnly(BasePermission):
    """Solo el dueño de la academia del profesor puede modificar."""

    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.academy.owner == request.user


class IsConvocatoriaCourseOwnerOrReadOnly(BasePermission):
    """Solo el dueño de la academia del curso de la convocatoria puede modificar."""

    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.course.academy.owner == request.user


class IsStudentOwnerOrReadOnly(BasePermission):
    """Solo el alumno dueño del recurso puede modificar."""

    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.student == request.user


class IsStaffOrReadOnly(BasePermission):
    """Solo staff escribe. Cualquiera lee."""

    def has_permission(self, request, view):
        if request.method in SAFE_METHODS:
            return True
        return request.user.is_authenticated and request.user.is_staff
```

---

# 9. Autenticación por API

### Archivo: `api/views/auth.py`

```python
from django.contrib.auth import authenticate, get_user_model, login, logout
from django.contrib.auth.password_validation import validate_password
from django.core.exceptions import ValidationError as DjangoValidationError
from rest_framework import status
from rest_framework.permissions import AllowAny, IsAuthenticated
from rest_framework.response import Response
```

```python
from rest_framework.views import APIView

from api.serializers.accounts import UserPublicSerializer

User = get_user_model()


class RegisterAPIView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        data = request.data
        username = data.get("username")
        email = data.get("email")
        password = data.get("password")
        is_academy = data.get("is_academy", False)
        is_student = data.get("is_student", False)

        if not username or not email or not password:
            return Response(
                {"detail": "username, email y password son obligatorios."},
                status=status.HTTP_400_BAD_REQUEST,
            )

        if User.objects.filter(username__iexact=username).exists():
            return Response(
                {"detail": "Ese nombre de usuario ya existe."},
                status=status.HTTP_400_BAD_REQUEST,
            )

        if User.objects.filter(email__iexact=email).exists():
            return Response(
                {"detail": "Ese email ya está registrado."},
                status=status.HTTP_400_BAD_REQUEST,
            )

        try:
            validate_password(password)
        except DjangoValidationError as e:
            return Response(
                {"detail": list(e.messages)},
                status=status.HTTP_400_BAD_REQUEST,
            )

        user = User.objects.create_user(
            username=username,
            email=email,
            password=password,
            is_academy=is_academy,
            is_student=is_student,
        )
```

```
        login(request, user)

        return Response(
            UserPublicSerializer(user).data,
            status=status.HTTP_201_CREATED,
        )


class LoginAPIView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        username = request.data.get("username")
        password = request.data.get("password")

        user = authenticate(request, username=username, password=password)

        if user is None:
            return Response(
                {"detail": "Credenciales incorrectas."},
                status=status.HTTP_401_UNAUTHORIZED,
            )

        login(request, user)

        return Response(UserPublicSerializer(user).data, status=status.HTTP_200_OK)


class LogoutAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        logout(request)
        return Response(status=status.HTTP_204_NO_CONTENT)


class MeAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response(UserPublicSerializer(request.user).data)

    def patch(self, request):
        user = request.user
        for field in ["first_name", "last_name", "city", "bio", "phone"]:
            if field in request.data:
                setattr(user, field, request.data[field])
        user.save()
        return Response(UserPublicSerializer(user).data)
```

```
class ChangePasswordAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        old_password = request.data.get("old_password")
        new_password = request.data.get("new_password")

        if not request.user.check_password(old_password):
            return Response(
                {"detail": "Contraseña actual incorrecta."},
                status=status.HTTP_400_BAD_REQUEST,
            )

        try:
            validate_password(new_password, user=request.user)
        except DjangoValidationError as e:
            return Response(
                {"detail": list(e.messages)},
                status=status.HTTP_400_BAD_REQUEST,
            )

        request.user.set_password(new_password)
        request.user.save()

        return Response(status=status.HTTP_204_NO_CONTENT)
```

# 10. Serializers por app

## 10.1. `api/serializers/accounts.py`

```
from django.contrib.auth import get_user_model
from rest_framework import serializers

User = get_user_model()


class UserPublicSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = [
            "id",
            "username",
            "first_name",
            "last_name",
            "city",
            "is_academy",
```

```
            "is_student",
            "is_verified",
        ]
        read_only_fields = fields
```

## 10.2. `api/serializers/academies.py`

```
from rest_framework import serializers

from academies.models import Academy, Teacher


class TeacherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Teacher
        fields = [
            "id",
            "academy",
            "first_name",
            "last_name",
            "email",
            "phone",
            "specialty",
            "bio",
            "photo",
            "is_active",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]


class AcademySerializer(serializers.ModelSerializer):
    owner = serializers.StringRelatedField(read_only=True)
    teachers_count = serializers.IntegerField(read_only=True)
    courses_count = serializers.IntegerField(read_only=True)
    favorite_count = serializers.IntegerField(read_only=True)

    class Meta:
        model = Academy
        fields = [
            "id",
            "owner",
            "name",
            "slug",
            "description",
            "city",
            "address",
            "website",
            "phone",
```

```
            "email",
            "logo",
            "is_verified",
            "is_active",
            "views_count",
            "teachers_count",
            "courses_count",
            "favorite_count",
            "created_at",
            "updated_at",
        ]
        read_only_fields = [
            "id",
            "owner",
            "slug",
            "views_count",
            "is_verified",
            "created_at",
            "updated_at",
        ]
```

## 10.3. `api/serializers/courses.py`

```
from rest_framework import serializers

from courses.models import (
    Category,
    Convocatoria,
    Course,
    CourseFavorite,
    CourseImage,
)


class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ["id", "name", "slug", "description", "icon", "is_active"]
        read_only_fields = ["id", "slug"]


class CourseImageSerializer(serializers.ModelSerializer):
    class Meta:
        model = CourseImage
        fields = ["id", "image", "alt_text", "is_primary", "position"]
        read_only_fields = ["id"]


class ConvocatoriaSerializer(serializers.ModelSerializer):
```

```
    class Meta:
        model = Convocatoria
        fields = [
            "id",
            "course",
            "name",
            "start_date",
            "end_date",
            "enrollment_deadline",
            "schedule",
            "modality",
            "location",
            "price",
            "deposit",
            "total_places",
            "reserved_places",
            "enrolled_places",
            "is_active",
            "created_at",
            "updated_at",
        ]
        read_only_fields = [
            "id",
            "reserved_places",
            "enrolled_places",
            "created_at",
            "updated_at",
        ]

    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError("El precio debe ser mayor que 0.")
        return value


class CourseSerializer(serializers.ModelSerializer):
    academy_name = serializers.CharField(source="academy.name", read_only=True)
    category_name = serializers.CharField(source="category.name", read_only=True)
    teacher_name = serializers.StringRelatedField(source="teacher", read_only=True)
    level_display = serializers.CharField(source="get_level_display", read_only=True)
    modality_display = serializers.CharField(source="get_modality_display", read_only=True)
    status_display = serializers.CharField(source="get_status_display", read_only=True)
    images = CourseImageSerializer(many=True, read_only=True)
    favorite_count = serializers.IntegerField(read_only=True)
    reviews_avg = serializers.FloatField(read_only=True)

    class Meta:
        model = Course
        fields = [
            "id",
            "academy",
```

```
            "academy_name",
            "category",
            "category_name",
            "teacher",
            "teacher_name",
            "title",
            "slug",
            "description",
            "objectives",
            "competences",
            "requirements",
            "level",
            "level_display",
            "modality",
            "modality_display",
            "status",
            "status_display",
            "enrollment_mode",
            "requires_level_test",
            "is_negotiable",
            "duration_hours",
            "views_count",
            "favorite_count",
            "reviews_avg",
            "images",
            "created_at",
            "updated_at",
        ]
        read_only_fields = [
            "id",
            "slug",
            "status",
            "views_count",
            "created_at",
            "updated_at",
        ]


class CourseFavoriteSerializer(serializers.ModelSerializer):
    course_detail = CourseSerializer(source="course", read_only=True)

    class Meta:
        model = CourseFavorite
        fields = ["id", "course", "course_detail", "created_at"]
        read_only_fields = ["id", "created_at"]
```

## 10.4. `api/serializers/enrollments.py`

```python
from rest_framework import serializers

from enrollments.models import (
    Enrollment,
    EnrollmentAnswer,
    EnrollmentDocument,
    WaitlistEntry,
)


class EnrollmentAnswerSerializer(serializers.ModelSerializer):
    class Meta:
        model = EnrollmentAnswer
        fields = ["id", "question_key", "question_text", "answer_text", "created_at"]
        read_only_fields = ["id", "created_at"]


class EnrollmentDocumentSerializer(serializers.ModelSerializer):
    class Meta:
        model = EnrollmentDocument
        fields = ["id", "name", "file", "is_verified", "uploaded_at"]
        read_only_fields = ["id", "is_verified", "uploaded_at"]


class EnrollmentSerializer(serializers.ModelSerializer):
    student_username = serializers.CharField(source="student.username", read_only=True)
    course_title = serializers.CharField(
        source="convocatoria.course.title", read_only=True
    )
    status_display = serializers.CharField(source="get_status_display", read_only=True)
    payment_status_display = serializers.CharField(
        source="get_payment_status_display", read_only=True
    )
    answers = EnrollmentAnswerSerializer(many=True, read_only=True)
    documents = EnrollmentDocumentSerializer(many=True, read_only=True)

    class Meta:
        model = Enrollment
        fields = [
            "id",
            "student",
            "student_username",
            "convocatoria",
            "course_title",
            "status",
            "status_display",
            "payment_status",
            "payment_status_display",
            "amount_paid",
            "discount_amount",
```

```
            "reservation_expires_at",
            "enrolled_at",
            "completed_at",
            "notes",
            "answers",
            "documents",
            "created_at",
            "updated_at",
        ]
        read_only_fields = [
            "id",
            "student",
            "payment_status",
            "amount_paid",
            "discount_amount",
            "enrolled_at",
            "completed_at",
            "created_at",
            "updated_at",
        ]


class WaitlistEntrySerializer(serializers.ModelSerializer):
    student_username = serializers.CharField(source="student.username", read_only=True)

    class Meta:
        model = WaitlistEntry
        fields = [
            "id",
            "student",
            "student_username",
            "convocatoria",
            "position",
            "is_notified",
            "notified_at",
            "converted_to_enrollment",
            "created_at",
        ]
        read_only_fields = [
            "id",
            "student",
            "position",
            "notified_at",
            "converted_to_enrollment",
            "created_at",
        ]
```

## 10.5. `api/serializers/reviews.py`

```python
from rest_framework import serializers

from reviews.models import AcademyReview, CourseReview


class CourseReviewSerializer(serializers.ModelSerializer):
    student_username = serializers.CharField(source="student.username", read_only=True)
    course_title = serializers.CharField(source="course.title", read_only=True)

    class Meta:
        model = CourseReview
        fields = [
            "id",
            "student",
            "student_username",
            "course",
            "course_title",
            "rating",
            "comment",
            "is_published",
            "created_at",
            "updated_at",
        ]
        read_only_fields = ["id", "student", "created_at", "updated_at"]

    def validate_rating(self, value):
        if value < 1 or value > 5:
            raise serializers.ValidationError("La puntuación debe estar entre 1 y 5.")
        return value


class AcademyReviewSerializer(serializers.ModelSerializer):
    student_username = serializers.CharField(source="student.username", read_only=True)
    academy_name = serializers.CharField(source="academy.name", read_only=True)

    class Meta:
        model = AcademyReview
        fields = [
            "id",
            "student",
            "student_username",
            "academy",
            "academy_name",
            "rating",
            "comment",
            "is_published",
            "created_at",
            "updated_at",
        ]
        read_only_fields = ["id", "student", "created_at", "updated_at"]
```

```
    def validate_rating(self, value):
        if value < 1 or value > 5:
            raise serializers.ValidationError("La puntuación debe estar entre 1 y 5.")
        return value
```

## 10.6. `api/serializers/payments.py`

```
from rest_framework import serializers

from payments.models import Discount, Payment, Scholarship


class DiscountSerializer(serializers.ModelSerializer):
    discount_type_display = serializers.CharField(
        source="get_discount_type_display", read_only=True
    )

    class Meta:
        model = Discount
        fields = [
            "id",
            "code",
            "description",
            "discount_type",
            "discount_type_display",
            "value",
            "valid_from",
            "valid_until",
            "is_active",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]


class PaymentSerializer(serializers.ModelSerializer):
    status_display = serializers.CharField(source="get_status_display", read_only=True)

    class Meta:
        model = Payment
        fields = [
            "id",
            "enrollment",
            "amount",
            "status",
            "status_display",
            "discount",
            "reference",
            "paid_at",
```

```
            "created_at",
        ]
        read_only_fields = ["id", "paid_at", "created_at"]


class ScholarshipSerializer(serializers.ModelSerializer):
    status_display = serializers.CharField(source="get_status_display", read_only=True)

    class Meta:
        model = Scholarship
        fields = [
            "id",
            "student",
            "enrollment",
            "status",
            "status_display",
            "amount",
            "reason",
            "reviewed_by",
            "reviewed_at",
            "created_at",
        ]
        read_only_fields = [
            "id",
            "student",
            "status",
            "reviewed_by",
            "reviewed_at",
            "created_at",
        ]
```

## 10.7. `api/serializers/notifications.py`

```
from rest_framework import serializers

from notifications.models import Notification


class NotificationSerializer(serializers.ModelSerializer):
    type_display = serializers.CharField(source="get_type_display", read_only=True)

    class Meta:
        model = Notification
        fields = [
            "id",
            "type",
            "type_display",
            "title",
            "message",
```

```
            "url",
            "is_read",
            "read_at",
            "created_at",
        ]
        read_only_fields = ["id", "read_at", "created_at"]
```

## 10.8. `api/serializers/evaluations.py`

```
from rest_framework import serializers

from evaluations.models import Evaluation


class EvaluationSerializer(serializers.ModelSerializer):
    kind_display = serializers.CharField(source="get_kind_display", read_only=True)

    class Meta:
        model = Evaluation
        fields = [
            "id",
            "enrollment",
            "kind",
            "kind_display",
            "score",
            "feedback",
            "evaluated_by",
            "evaluated_at",
            "created_at",
        ]
        read_only_fields = [
            "id",
            "evaluated_by",
            "evaluated_at",
            "created_at",
        ]
```

## 10.9. `api/serializers/talent.py`

```
from rest_framework import serializers

from talent.models import Competence, PassportEntry, TalentPassport


class CompetenceSerializer(serializers.ModelSerializer):
    class Meta:
        model = Competence
        fields = ["id", "name", "description", "category", "is_active", "created_at"]
```

```
        read_only_fields = ["id", "created_at"]


class PassportEntrySerializer(serializers.ModelSerializer):
    competence_name = serializers.CharField(source="competence.name", read_only=True)

    class Meta:
        model = PassportEntry
        fields = [
            "id",
            "passport",
            "competence",
            "competence_name",
            "level",
            "evidence",
            "obtained_at",
        ]
        read_only_fields = ["id", "obtained_at"]

    def validate_level(self, value):
        if value < 1 or value > 5:
            raise serializers.ValidationError("El nivel debe estar entre 1 y 5.")
        return value


class TalentPassportSerializer(serializers.ModelSerializer):
    student_username = serializers.CharField(source="student.username", read_only=True)
    entries = PassportEntrySerializer(many=True, read_only=True)

    class Meta:
        model = TalentPassport
        fields = [
            "id",
            "student",
            "student_username",
            "headline",
            "summary",
            "entries",
            "created_at",
            "updated_at",
        ]
        read_only_fields = ["id", "student", "created_at", "updated_at"]
```

## 10.10. `api/serializers/level_tests.py`

```
from rest_framework import serializers

from level_tests.models import LevelTest, LevelTestAttempt, LevelTestQuestion
```

```python
class LevelTestQuestionSerializer(serializers.ModelSerializer):
    class Meta:
        model = LevelTestQuestion
        fields = [
            "id",
            "question",
            "option_a",
            "option_b",
            "option_c",
            "option_d",
        ]
        read_only_fields = fields


class LevelTestSerializer(serializers.ModelSerializer):
    questions = LevelTestQuestionSerializer(many=True, read_only=True)

    class Meta:
        model = LevelTest
        fields = [
            "id",
            "course",
            "title",
            "description",
            "passing_score",
            "is_active",
            "questions",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]


class LevelTestAttemptSerializer(serializers.ModelSerializer):
    student_username = serializers.CharField(source="student.username", read_only=True)

    class Meta:
        model = LevelTestAttempt
        fields = [
            "id",
            "student",
            "student_username",
            "level_test",
            "score",
            "passed",
            "started_at",
            "finished_at",
        ]
        read_only_fields = ["id", "student", "started_at"]
```

# 11. Filtros por app

## 11.1. `api/filters/courses.py`

```
import django_filters

from courses.models import Course


class CourseFilter(django_filters.FilterSet):
    min_price = django_filters.NumberFilter(
        field_name="convocatorias__price",
        lookup_expr="gte",
    )
    max_price = django_filters.NumberFilter(
        field_name="convocatorias__price",
        lookup_expr="lte",
    )

    class Meta:
        model = Course
        fields = [
            "category",
            "academy",
            "level",
            "modality",
            "status",
            "enrollment_mode",
            "is_negotiable",
            "requires_level_test",
        ]
```

## 11.2. `api/filters/academies.py`

```
import django_filters

from academies.models import Academy


class AcademyFilter(django_filters.FilterSet):
    class Meta:
        model = Academy
        fields = ["city", "is_verified", "is_active"]
```

## 11.3. `api/filters/enrollments.py`

```python
import django_filters

from enrollments.models import Enrollment


class EnrollmentFilter(django_filters.FilterSet):
    class Meta:
        model = Enrollment
        fields = ["status", "payment_status", "convocatoria", "student"]
```

### 11.4. `api/filters/payments.py`

```python
import django_filters

from payments.models import Payment


class PaymentFilter(django_filters.FilterSet):
    class Meta:
        model = Payment
        fields = ["status", "enrollment"]
```

---

## 12. ViewSets por app

### 12.1. `api/views/academies.py`

```python
from django.db.models import Count
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from academies.models import Academy, AcademyFavorite, Teacher
from api.filters.academies import AcademyFilter
from api.permissions import (
    IsAcademyOwnerOrReadOnly,
    IsTeacherAcademyOwnerOrReadOnly,
)
from api.serializers.academies import AcademySerializer, TeacherSerializer
from api.serializers.courses import CourseSerializer


class AcademyViewSet(viewsets.ModelViewSet):
    serializer_class = AcademySerializer
    permission_classes = [IsAcademyOwnerOrReadOnly]
```

```
    filterset_class = AcademyFilter
    search_fields = ["name", "description", "city"]
    ordering_fields = ["created_at", "name", "views_count"]
    ordering = ["-created_at"]
    lookup_field = "slug"

    def get_queryset(self):
        return (
            Academy.objects
            .filter(is_active=True)
            .select_related("owner")
            .annotate(
                teachers_count=Count("teachers", distinct=True),
                courses_count=Count("courses", distinct=True),
                favorite_count=Count("favorites", distinct=True),
            )
        )

    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)

    @action(detail=False, methods=["get"], permission_classes=[IsAuthenticated])
    def mine(self, request):
        queryset = self.get_queryset().filter(owner=request.user)
        page = self.paginate_queryset(queryset)
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
            )
        return Response(self.get_serializer(queryset, many=True).data)

    @action(detail=True, methods=["get"])
    def teachers(self, request, slug=None):
        academy = self.get_object()
        teachers = academy.teachers.filter(is_active=True)
        return Response(TeacherSerializer(teachers, many=True).data)

    @action(detail=True, methods=["get"])
    def courses(self, request, slug=None):
        academy = self.get_object()
        courses = academy.courses.filter(status="published")
        page = self.paginate_queryset(courses)
        if page is not None:
            return self.get_paginated_response(
                CourseSerializer(page, many=True).data
            )
        return Response(CourseSerializer(courses, many=True).data)

    @action(detail=True, methods=["post"], permission_classes=[IsAuthenticated])
    def favorite(self, request, slug=None):
        academy = self.get_object()
```

```
        if academy.owner == request.user:
            return Response(
                {"detail": "No puedes guardar tu propia academia."},
                status=400,
            )

        favorite, created = AcademyFavorite.objects.get_or_create(
            user=request.user,
            academy=academy,
        )

        if created:
            return Response({"favorited": True}, status=201)

        favorite.delete()
        return Response({"favorited": False}, status=200)


class TeacherViewSet(viewsets.ModelViewSet):
    serializer_class = TeacherSerializer
    permission_classes = [IsTeacherAcademyOwnerOrReadOnly]

    def get_queryset(self):
        return Teacher.objects.select_related("academy").filter(is_active=True)
```

## 12.2. `api/views/courses.py`

```
from django.db.models import Avg, Count
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from api.filters.courses import CourseFilter
from api.permissions import IsCourseAcademyOwnerOrReadOnly
from api.serializers.courses import (
    CategorySerializer,
    CourseFavoriteSerializer,
    CourseSerializer,
)
from courses.models import Category, Course, CourseFavorite
from courses.services import archive_course


class CategoryViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = CategorySerializer
    lookup_field = "slug"
```

```python
    def get_queryset(self):
        return Category.objects.filter(is_active=True).order_by("name")


class CourseViewSet(viewsets.ModelViewSet):
    serializer_class = CourseSerializer
    permission_classes = [IsCourseAcademyOwnerOrReadOnly]
    filterset_class = CourseFilter
    search_fields = [
        "title",
        "description",
        "objectives",
        "competences",
        "academy__name",
        "category__name",
    ]
    ordering_fields = [
        "created_at",
        "views_count",
        "duration_hours",
        "favorite_count",
        "reviews_avg",
    ]
    ordering = ["-created_at"]
    lookup_field = "pk"

    def get_queryset(self):
        return (
            Course.objects
            .filter(status="published")
            .select_related("academy", "category", "teacher")
            .prefetch_related("images", "convocatorias")
            .annotate(
                favorite_count=Count("favorites", distinct=True),
                reviews_avg=Avg("reviews__rating"),
            )
        )

    def perform_create(self, serializer):
        serializer.save(academy=self.request.user.academies.first())

    def perform_destroy(self, instance):
        archive_course(instance.pk, self.request.user)

    @action(detail=False, methods=["get"], permission_classes=[IsAuthenticated])
    def mine(self, request):
        queryset = (
            Course.objects
            .filter(academy__owner=request.user)
            .select_related("academy", "category", "teacher")
            .annotate(
```

```
                favorite_count=Count("favorites", distinct=True),
                reviews_avg=Avg("reviews__rating"),
            )
        )
        page = self.paginate_queryset(queryset)
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
            )
        return Response(self.get_serializer(queryset, many=True).data)

    @action(detail=True, methods=["get"])
    def similar(self, request, pk=None):
        course = self.get_object()
        similar = self.get_queryset().filter(category=course.category).exclude(pk=course.pk)
        page = self.paginate_queryset(similar)
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
            )
        return Response(self.get_serializer(similar, many=True).data)

    @action(detail=True, methods=["post"], permission_classes=[IsAuthenticated])
    def favorite(self, request, pk=None):
        course = self.get_object()

        if course.academy.owner == request.user:
            return Response(
                {"detail": "No puedes guardar tu propio curso."},
                status=400,
            )

        favorite, created = CourseFavorite.objects.get_or_create(
            user=request.user,
            course=course,
        )

        if created:
            return Response({"favorited": True}, status=201)

        favorite.delete()
        return Response({"favorited": False}, status=200)


class CourseFavoriteViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = CourseFavoriteSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return (
            CourseFavorite.objects
```

```
            .filter(user=self.request.user)
            .select_related("course", "course__academy", "course__category")
            .prefetch_related("course__images")
        )
```

## 12.3. `api/views/enrollments.py`

```
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from api.filters.enrollments import EnrollmentFilter
from api.permissions import IsStudentOwnerOrReadOnly
from api.serializers.enrollments import EnrollmentSerializer
from enrollments.models import Enrollment
from enrollments.services import reserve_place


class EnrollmentViewSet(viewsets.ModelViewSet):
    serializer_class = EnrollmentSerializer
    permission_classes = [IsAuthenticated, IsStudentOwnerOrReadOnly]
    filterset_class = EnrollmentFilter

    def get_queryset(self):
        return (
            Enrollment.objects
            .select_related("student", "convocatoria", "convocatoria__course")
            .prefetch_related("answers", "documents")
            .order_by("-created_at")
        )

    def perform_create(self, serializer):
        serializer.save(student=self.request.user)

    @action(detail=False, methods=["get"])
    def mine(self, request):
        queryset = self.get_queryset().filter(student=request.user)
        page = self.paginate_queryset(queryset)
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
            )
        return Response(self.get_serializer(queryset, many=True).data)

    @action(detail=False, methods=["get"])
    def academy(self, request):
        queryset = self.get_queryset().filter(
            convocatoria__course__academy__owner=request.user
```

```
        )
        page = self.paginate_queryset(queryset)
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
            )
        return Response(self.get_serializer(queryset, many=True).data)

    @action(detail=True, methods=["post"])
    def approve(self, request, pk=None):
        enrollment = self.get_object()

        if enrollment.convocatoria.course.academy.owner != request.user:
            return Response(
                {"detail": "No tienes permiso."},
                status=403,
            )

        enrollment.status = "approved"
        enrollment.save(update_fields=["status"])
        return Response(self.get_serializer(enrollment).data)

    @action(detail=True, methods=["post"])
    def enroll(self, request, pk=None):
        enrollment = self.get_object()

        if enrollment.convocatoria.course.academy.owner != request.user:
            return Response(
                {"detail": "No tienes permiso."},
                status=403,
            )

        enrollment.status = "enrolled"
        enrollment.save(update_fields=["status"])
        return Response(self.get_serializer(enrollment).data)


class ConvocatoriaReserveViewSet(viewsets.ViewSet):
    permission_classes = [IsAuthenticated]

    @action(detail=True, methods=["post"])
    def reserve(self, request, pk=None):
        try:
            enrollment = reserve_place(request.user, pk)
            return Response(
                EnrollmentSerializer(enrollment).data,
                status=201,
            )
        except ValueError as e:
            return Response({"detail": str(e)}, status=400)
```

## 12.4. `api/views/reviews.py`

```
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated

from api.permissions import IsStudentOwnerOrReadOnly
from api.serializers.reviews import AcademyReviewSerializer, CourseReviewSerializer
from reviews.models import AcademyReview, CourseReview


class CourseReviewViewSet(viewsets.ModelViewSet):
    serializer_class = CourseReviewSerializer
    permission_classes = [IsStudentOwnerOrReadOnly]
    filterset_fields = ["course", "rating", "is_published"]

    def get_queryset(self):
        return (
            CourseReview.objects
            .filter(is_published=True)
            .select_related("student", "course")
            .order_by("-created_at")
        )

    def perform_create(self, serializer):
        serializer.save(student=self.request.user)


class AcademyReviewViewSet(viewsets.ModelViewSet):
    serializer_class = AcademyReviewSerializer
    permission_classes = [IsStudentOwnerOrReadOnly]
    filterset_fields = ["academy", "rating", "is_published"]

    def get_queryset(self):
        return (
            AcademyReview.objects
            .filter(is_published=True)
            .select_related("student", "academy")
            .order_by("-created_at")
        )

    def perform_create(self, serializer):
        serializer.save(student=self.request.user)


class MyReviewViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = CourseReviewSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return (
```

```
            CourseReview.objects
            .filter(student=self.request.user)
            .select_related("course")
            .order_by("-created_at")
        )
```

## 12.5. `api/views/payments.py`

```
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from api.serializers.payments import (
    DiscountSerializer,
    PaymentSerializer,
    ScholarshipSerializer,
)
from payments.models import Discount, Payment, Scholarship


class DiscountViewSet(viewsets.ModelViewSet):
    serializer_class = DiscountSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        if self.request.user.is_staff:
            return Discount.objects.all().order_by("-created_at")
        return Discount.objects.filter(is_active=True).order_by("-created_at")


class PaymentViewSet(viewsets.ModelViewSet):
    serializer_class = PaymentSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        if self.request.user.is_academy:
            return Payment.objects.filter(
                enrollment__convocatoria__course__academy__owner=self.request.user
            ).order_by("-created_at")
        return Payment.objects.filter(enrollment__student=self.request.user).order_by("-created_a

    @action(detail=False, methods=["get"])
    def mine(self, request):
        queryset = Payment.objects.filter(enrollment__student=request.user)
        page = self.paginate_queryset(queryset)
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
```

```
            )
        return Response(self.get_serializer(queryset, many=True).data)


class ScholarshipViewSet(viewsets.ModelViewSet):
    serializer_class = ScholarshipSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        if self.request.user.is_academy:
            return Scholarship.objects.filter(
                enrollment__convocatoria__course__academy__owner=self.request.user
            ).order_by("-created_at")
        return Scholarship.objects.filter(student=self.request.user).order_by("-created_at")

    def perform_create(self, serializer):
        serializer.save(student=self.request.user)

    @action(detail=True, methods=["post"])
    def approve(self, request, pk=None):
        scholarship = self.get_object()
        if not request.user.is_academy:
            return Response({"detail": "Solo academias."}, status=403)
        scholarship.status = Scholarship.Status.APPROVED
        scholarship.save()
        return Response(self.get_serializer(scholarship).data)

    @action(detail=True, methods=["post"])
    def reject(self, request, pk=None):
        scholarship = self.get_object()
        if not request.user.is_academy:
            return Response({"detail": "Solo academias."}, status=403)
        scholarship.status = Scholarship.Status.REJECTED
        scholarship.save()
        return Response(self.get_serializer(scholarship).data)
```

## 12.6. `api/views/notifications.py`

```
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from api.serializers.notifications import NotificationSerializer
from notifications.models import Notification


class NotificationViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = NotificationSerializer
```

```
    permission_classes = [IsAuthenticated]
    filterset_fields = ["is_read", "type"]

    def get_queryset(self):
        return Notification.objects.filter(user=self.request.user).order_by("-created_at")

    @action(detail=True, methods=["post"])
    def read(self, request, pk=None):
        notification = self.get_object()
        notification.is_read = True
        notification.save(update_fields=["is_read"])
        return Response(self.get_serializer(notification).data)

    @action(detail=False, methods=["post"])
    def read_all(self, request):
        Notification.objects.filter(user=request.user, is_read=False).update(
            is_read=True
        )
        return Response(status=204)
```

## 12.7. `api/views/evaluations.py`

```
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from api.serializers.evaluations import EvaluationSerializer
from evaluations.models import Evaluation


class EvaluationViewSet(viewsets.ModelViewSet):
    serializer_class = EvaluationSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        if self.request.user.is_academy:
            return Evaluation.objects.filter(
                enrollment__convocatoria__course__academy__owner=self.request.user
            ).order_by("-created_at")
        return Evaluation.objects.filter(enrollment__student=self.request.user).order_by("-create

    def perform_create(self, serializer):
        serializer.save(evaluated_by=self.request.user)

    @action(detail=False, methods=["get"])
    def mine(self, request):
        queryset = Evaluation.objects.filter(enrollment__student=request.user)
        page = self.paginate_queryset(queryset)
```

```
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
            )
        return Response(self.get_serializer(queryset, many=True).data)

    @action(detail=False, methods=["get"])
    def academy(self, request):
        queryset = Evaluation.objects.filter(
            enrollment__convocatoria__course__academy__owner=request.user
        )
        page = self.paginate_queryset(queryset)
        if page is not None:
            return self.get_paginated_response(
                self.get_serializer(page, many=True).data
            )
        return Response(self.get_serializer(queryset, many=True).data)
```

## 12.8. `api/views/talent.py`

```
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from api.permissions import IsStaffOrReadOnly
from api.serializers.talent import (
    CompetenceSerializer,
    PassportEntrySerializer,
    TalentPassportSerializer,
)
from talent.models import Competence, PassportEntry, TalentPassport


class CompetenceViewSet(viewsets.ModelViewSet):
    serializer_class = CompetenceSerializer
    permission_classes = [IsStaffOrReadOnly]
    search_fields = ["name", "description"]

    def get_queryset(self):
        return Competence.objects.filter(is_active=True).order_by("name")


class TalentPassportViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = TalentPassportSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return TalentPassport.objects.filter(student=self.request.user)
```

```
    @action(detail=False, methods=["get"])
    def mine(self, request):
        passport, _ = TalentPassport.objects.get_or_create(student=request.user)
        return Response(TalentPassportSerializer(passport).data)


class PassportEntryViewSet(viewsets.ModelViewSet):
    serializer_class = PassportEntrySerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return PassportEntry.objects.filter(
            passport__student=self.request.user
        ).select_related("competence").order_by("-obtained_at")
```

## 12.9. `api/views/level_tests.py`

```
from django.utils import timezone
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from api.serializers.level_tests import (
    LevelTestAttemptSerializer,
    LevelTestSerializer,
)
from level_tests.models import LevelTest, LevelTestAttempt


class LevelTestViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = LevelTestSerializer
    filterset_fields = ["course", "is_active"]
    search_fields = ["title", "description"]

    def get_queryset(self):
        return LevelTest.objects.filter(is_active=True).select_related("course")

    @action(detail=True, methods=["post"], permission_classes=[IsAuthenticated])
    def attempt(self, request, pk=None):
        level_test = self.get_object()

        score = request.data.get("score", 0)
        passed = score >= level_test.passing_score

        attempt = LevelTestAttempt.objects.create(
            student=request.user,
            level_test=level_test,
```

```
            score=score,
            passed=passed,
            finished_at=timezone.now(),
        )

        return Response(LevelTestAttemptSerializer(attempt).data, status=201)

    @action(detail=True, methods=["get"], permission_classes=[IsAuthenticated])
    def attempts_mine(self, request, pk=None):
        level_test = self.get_object()
        attempts = LevelTestAttempt.objects.filter(
            student=request.user,
            level_test=level_test,
        ).order_by("-started_at")
        return Response(LevelTestAttemptSerializer(attempts, many=True).data)
```

## 12.10. `api/views/stats.py`

```
from django.contrib.auth import get_user_model
from django.db.models import Avg, Count
from rest_framework.response import Response
from rest_framework.views import APIView

from courses.models import Convocatoria, Course
from enrollments.models import Enrollment
from reviews.models import CourseReview

User = get_user_model()


class StatsAPIView(APIView):
    def get(self, request):
        total_courses = Course.objects.count()
        active_courses = Course.objects.filter(status="published").count()
        total_academies = User.objects.filter(is_academy=True).count()
        total_students = User.objects.filter(is_student=True).count()
        total_enrollments = Enrollment.objects.count()
        total_reviews = CourseReview.objects.count()
        avg_rating = CourseReview.objects.aggregate(avg=Avg("rating"))["avg"] or 0
        avg_price = Convocatoria.objects.filter(is_active=True).aggregate(
            avg=Avg("price")
        )["avg"] or 0

        return Response(
            {
                "courses": total_courses,
                "active_courses": active_courses,
                "academies": total_academies,
                "students": total_students,
```

```
                "enrollments": total_enrollments,
                "reviews": total_reviews,
                "avg_rating": round(float(avg_rating), 2),
                "avg_price": str(avg_price),
            }
        )
```

### 12.11. `api/views/accounts.py`

```
from django.contrib.auth import get_user_model
from rest_framework import viewsets
from rest_framework.permissions import IsAdminUser

from api.serializers.accounts import UserPublicSerializer

User = get_user_model()


class UserViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = UserPublicSerializer
    permission_classes = [IsAdminUser]
    search_fields = ["username", "email", "first_name", "last_name"]

    def get_queryset(self):
        return User.objects.all().order_by("-created_at")
```

---

## 13. Acciones personalizadas

Ya incluidas en los ViewSets de arriba:

| Endpoint | Acción | Permiso |
|---|---|---|
| `GET /api/v1/academies/mine/` | Mis academias | `IsAuthenticated` |
| `GET /api/v1/academies/{slug}/teachers/` | Profesores de academia | Público |
| `GET /api/v1/academies/{slug}/courses/` | Cursos de academia | Público |
| `POST /api/v1/academies/{slug}/favorite/` | Alternar favorito | `IsAuthenticated` |
| `GET /api/v1/courses/mine/` | Mis cursos | `IsAuthenticated` |
| `GET /api/v1/courses/{pk}/similar/` | Cursos similares | Público |

| Endpoint | Acción | Permiso |
|---|---|---|
| `POST /api/v1/courses/{pk}/favorite/` | Alternar favorito | `IsAuthenticated` |
| `GET /api/v1/enrollments/mine/` | Mis inscripciones | `IsAuthenticated` |
| `GET /api/v1/enrollments/academy/` | Inscripciones de mi academia | `IsAuthenticated` |
| `POST /api/v1/enrollments/{pk}/approve/` | Aprobar inscripción | Academia |
| `POST /api/v1/enrollments/{pk}/enroll/` | Matricular | Academia |
| `POST /api/v1/convocatorias/{pk}/reserve/` | Reservar plaza | `IsAuthenticated` |
| `GET /api/v1/payments/mine/` | Mis pagos | `IsAuthenticated` |
| `POST /api/v1/scholarships/{pk}/approve/` | Aprobar beca | Academia |
| `POST /api/v1/scholarships/{pk}/reject/` | Rechazar beca | Academia |
| `POST /api/v1/notifications/{pk}/read/` | Marcar como leída | `IsAuthenticated` |
| `POST /api/v1/notifications/read_all/` | Marcar todas como leídas | `IsAuthenticated` |
| `GET /api/v1/evaluations/mine/` | Mis evaluaciones | `IsAuthenticated` |
| `GET /api/v1/evaluations/academy/` | Evaluaciones de mi academia | `IsAuthenticated` |
| `GET /api/v1/passport/mine/` | Mi pasaporte | `IsAuthenticated` |
| `POST /api/v1/level-tests/{pk}/attempt/` | Realizar prueba | `IsAuthenticated` |
| `GET /api/v1/level-tests/{pk}/attempts_mine/` | Mis intentos | `IsAuthenticated` |

---

# 14. Routers

**Archivo:** `api/urls.py`

```python
from django.urls import include, path
from rest_framework.routers import DefaultRouter

from api.views.accounts import UserViewSet
from api.views.academies import AcademyViewSet, TeacherViewSet
from api.views.auth import (
    ChangePasswordAPIView,
    LoginAPIView,
    LogoutAPIView,
    MeAPIView,
    RegisterAPIView,
)
from api.views.courses import (
    CategoryViewSet,
    CourseFavoriteViewSet,
    CourseViewSet,
)
from api.views.enrollments import (
    ConvocatoriaReserveViewSet,
    EnrollmentViewSet,
)
from api.views.evaluations import EvaluationViewSet
from api.views.level_tests import LevelTestViewSet
from api.views.notifications import NotificationViewSet
from api.views.payments import (
    DiscountViewSet,
    PaymentViewSet,
    ScholarshipViewSet,
)
from api.views.reviews import (
    AcademyReviewViewSet,
    CourseReviewViewSet,
    MyReviewViewSet,
)
from api.views.stats import StatsAPIView
from api.views.talent import (
    CompetenceViewSet,
    PassportEntryViewSet,
    TalentPassportViewSet,
)

app_name = "api"

router = DefaultRouter()
router.register("users", UserViewSet, basename="user")
router.register("academies", AcademyViewSet, basename="academy")
router.register("teachers", TeacherViewSet, basename="teacher")
router.register("courses", CourseViewSet, basename="course")
router.register("categories", CategoryViewSet, basename="category")
router.register("course-favorites", CourseFavoriteViewSet, basename="course-favorite")
```

```
router.register("enrollments", EnrollmentViewSet, basename="enrollment")
router.register("convocatorias", ConvocatoriaReserveViewSet, basename="convocatoria")
router.register("course-reviews", CourseReviewViewSet, basename="course-review")
router.register("academy-reviews", AcademyReviewViewSet, basename="academy-review")
router.register("my-reviews", MyReviewViewSet, basename="my-review")
router.register("discounts", DiscountViewSet, basename="discount")
router.register("payments", PaymentViewSet, basename="payment")
router.register("scholarships", ScholarshipViewSet, basename="scholarship")
router.register("notifications", NotificationViewSet, basename="notification")
router.register("evaluations", EvaluationViewSet, basename="evaluation")
router.register("competences", CompetenceViewSet, basename="competence")
router.register("passport", TalentPassportViewSet, basename="passport")
router.register("passport-entries", PassportEntryViewSet, basename="passport-entry")
router.register("level-tests", LevelTestViewSet, basename="level-test")

urlpatterns = [
    # Autenticación
    path("auth/register/", RegisterAPIView.as_view(), name="auth-register"),
    path("auth/login/", LoginAPIView.as_view(), name="auth-login"),
    path("auth/logout/", LogoutAPIView.as_view(), name="auth-logout"),
    path("auth/me/", MeAPIView.as_view(), name="auth-me"),
    path(
        "auth/change-password/",
        ChangePasswordAPIView.as_view(),
        name="auth-change-password",
    ),

    # Stats
    path("stats/", StatsAPIView.as_view(), name="stats"),
]

urlpatterns += router.urls
```

---

# 15. Documentación OpenAPI + Swagger + ReDoc

Con `drf-spectacular` ya configurado en `base.py` y las rutas añadidas en `config/urls.py`, los endpoints están disponibles automáticamente en:

- `/api/schema/` → esquema OpenAPI en YAML.
- `/api/docs/` → Swagger UI.
- `/api/redoc/` → ReDoc.

Para documentar mejor las acciones personalizadas, añade `@extend_schema`:

```
from drf_spectacular.utils import extend_schema
```

```
@extend_schema(
    summary="Alternar favorito de curso",
    responses={201: None, 200: None, 400: None},
)
@action(detail=True, methods=["post"], permission_classes=[IsAuthenticated])
def favorite(self, request, pk=None):
    ...
```

Con esto, Swagger mostrará el resumen y los códigos de respuesta de cada acción.

---

# 16. Tests por app

## 16.1. `api/tests/test_auth.py`

```
from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

User = get_user_model()


class AuthTests(APITestCase):

    def test_register_creates_user(self):
        response = self.client.post(
            "/api/v1/auth/register/",
            {
                "username": "laura",
                "email": "laura@example.com",
                "password": "TestPassword123!",
                "is_student": True,
            },
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        self.assertTrue(User.objects.filter(username="laura").exists())

    def test_login_returns_user(self):
        User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
        )
        response = self.client.post(
            "/api/v1/auth/login/",
            {"username": "laura", "password": "TestPassword123!"},
            format="json",
```

```
        )
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertEqual(response.data["username"], "laura")

    def test_me_requires_auth(self):
        response = self.client.get("/api/v1/auth/me/")
        self.assertIn(
            response.status_code,
            [status.HTTP_401_UNAUTHORIZED, status.HTTP_403_FORBIDDEN],
        )

    def test_me_returns_user_when_authenticated(self):
        user = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
        )
        self.client.force_authenticate(user=user)
        response = self.client.get("/api/v1/auth/me/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertEqual(response.data["username"], "laura")
        self.assertNotIn("email", response.data)
        self.assertNotIn("phone", response.data)
```

## 16.2. `api/tests/test_courses.py`

```
from decimal import Decimal

from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

from academies.models import Academy
from courses.models import Category, Course

User = get_user_model()


class CourseAPITests(APITestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.other = User.objects.create_user(
            username="codeacademy",
```

```
            email="info@codeacademy.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(
            name="Idiomas",
            slug="idiomas",
        )
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status="published",
        )

    def test_list_is_public(self):
        response = self.client.get("/api/v1/courses/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_detail_is_public(self):
        response = self.client.get(f"/api/v1/courses/{self.course.pk}/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_anonymous_cannot_create(self):
        response = self.client.post(
            "/api/v1/courses/",
            {
                "academy": self.academy.pk,
                "category": self.category.pk,
                "title": "Francés B1",
                "description": "...",
            },
            format="json",
        )
        self.assertIn(
            response.status_code,
            [status.HTTP_401_UNAUTHORIZED, status.HTTP_403_FORBIDDEN],
        )

    def test_owner_can_create(self):
        self.client.force_authenticate(user=self.owner)
        response = self.client.post(
            "/api/v1/courses/",
            {
```

```
                "academy": self.academy.pk,
                "category": self.category.pk,
                "title": "Francés B1",
                "description": "...",
            },
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

    def test_other_user_cannot_patch(self):
        self.client.force_authenticate(user=self.other)
        response = self.client.patch(
            f"/api/v1/courses/{self.course.pk}/",
            {"title": "Hackeado"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_403_FORBIDDEN)

    def test_search_works(self):
        response = self.client.get("/api/v1/courses/?search=Inglés")
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertGreaterEqual(response.data["count"], 1)

    def test_filter_by_category(self):
        response = self.client.get(f"/api/v1/courses/?category={self.category.pk}")
        self.assertEqual(response.status_code, status.HTTP_200_OK)
```

## 16.3. `api/tests/test_academies.py`

```
from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

from academies.models import Academy

User = get_user_model()


class AcademyAPITests(APITestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
```

```
            name="English Now",
            slug="english-now",
            city="Madrid",
        )

    def test_list_is_public(self):
        response = self.client.get("/api/v1/academies/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_detail_by_slug(self):
        response = self.client.get("/api/v1/academies/english-now/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertEqual(response.data["name"], "English Now")

    def test_mine_requires_auth(self):
        response = self.client.get("/api/v1/academies/mine/")
        self.assertIn(
            response.status_code,
            [status.HTTP_401_UNAUTHORIZED, status.HTTP_403_FORBIDDEN],
        )

    def test_mine_returns_own_academies(self):
        self.client.force_authenticate(user=self.owner)
        response = self.client.get("/api/v1/academies/mine/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_search_by_city(self):
        response = self.client.get("/api/v1/academies/?city=Madrid")
        self.assertEqual(response.status_code, status.HTTP_200_OK)
```

## 16.4. `api/tests/test_enrollments.py`

```
from decimal import Decimal

from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

from academies.models import Academy
from courses.models import Category, Convocatoria, Course

User = get_user_model()


class EnrollmentAPITests(APITestCase):

    def setUp(self):
        self.academy_user = User.objects.create_user(
            username="englishnow",
```

```
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.academy_user,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status="published",
        )
        self.convocatoria = Convocatoria.objects.create(
            course=self.course,
            name="Verano",
            start_date="2026-07-01",
            end_date="2026-09-30",
            price=Decimal("480.00"),
            total_places=15,
        )

    def test_student_can_create_enrollment(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.post(
            "/api/v1/enrollments/",
            {"convocatoria": self.convocatoria.pk},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

    def test_anonymous_cannot_create(self):
        response = self.client.post(
            "/api/v1/enrollments/",
            {"convocatoria": self.convocatoria.pk},
            format="json",
        )
        self.assertIn(
            response.status_code,
            [status.HTTP_401_UNAUTHORIZED, status.HTTP_403_FORBIDDEN],
```

```
        )

    def test_mine_returns_own(self):
        self.client.force_authenticate(user=self.student)
        self.client.post(
            "/api/v1/enrollments/",
            {"convocatoria": self.convocatoria.pk},
            format="json",
        )
        response = self.client.get("/api/v1/enrollments/mine/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_reserve_via_action(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.post(
            f"/api/v1/convocatorias/{self.convocatoria.pk}/reserve/",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
```

## 16.5. `api/tests/test_reviews.py`

```
from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

from academies.models import Academy
from courses.models import Category, Course
from reviews.models import CourseReview

User = get_user_model()


class ReviewAPITests(APITestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
```

```
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status="published",
        )

    def test_list_is_public(self):
        response = self.client.get("/api/v1/course-reviews/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_student_can_create(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.post(
            "/api/v1/course-reviews/",
            {"course": self.course.pk, "rating": 5, "comment": "Excelente"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

    def test_rating_must_be_between_1_and_5(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.post(
            "/api/v1/course-reviews/",
            {"course": self.course.pk, "rating": 6, "comment": "..."},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_400_BAD_REQUEST)
```

## 16.6. `api/tests/test_favorites.py`

```
from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

from academies.models import Academy
from courses.models import Category, Course, CourseFavorite

User = get_user_model()


class CourseFavoriteAPITests(APITestCase):
```

```python
    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status="published",
        )

    def test_student_can_favorite(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.post(
            f"/api/v1/courses/{self.course.pk}/favorite/",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        self.assertTrue(
            CourseFavorite.objects.filter(
                user=self.student,
                course=self.course,
            ).exists()
        )

    def test_owner_cannot_favorite_own_course(self):
        self.client.force_authenticate(user=self.owner)
        response = self.client.post(
            f"/api/v1/courses/{self.course.pk}/favorite/",
        )
        self.assertEqual(response.status_code, status.HTTP_400_BAD_REQUEST)

    def test_anonymous_cannot_favorite(self):
        response = self.client.post(
            f"/api/v1/courses/{self.course.pk}/favorite/",
```

```
        )
        self.assertIn(
            response.status_code,
            [status.HTTP_401_UNAUTHORIZED, status.HTTP_403_FORBIDDEN],
        )
```

## 16.7. `api/tests/test_talent.py`

```
from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

from talent.models import TalentPassport

User = get_user_model()


class TalentAPITests(APITestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )

    def test_passport_mine_creates_on_demand(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.get("/api/v1/passport/mine/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertTrue(
            TalentPassport.objects.filter(student=self.student).exists()
        )
```

## 16.8. `api/tests/test_stats.py`

```
from rest_framework import status
from rest_framework.test import APITestCase


class StatsAPITests(APITestCase):

    def test_stats_is_public(self):
        response = self.client.get("/api/v1/stats/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertIn("courses", response.data)
        self.assertIn("academies", response.data)
```

# 17. Ejemplos con curl

## Listar cursos

```
curl "http://127.0.0.1:8000/api/v1/courses/"
```

## Buscar y filtrar

```
curl "http://127.0.0.1:8000/api/v1/courses/?search=Inglés&ordering=-views_count"
```

## Login

```
curl -X POST "http://127.0.0.1:8000/api/v1/auth/login/" \
    -H "Content-Type: application/json" \
    -d '{"username": "laura", "password": "TestPassword123!"}' \
    -c cookies.txt
```

## Ver mi perfil

```
curl "http://127.0.0.1:8000/api/v1/auth/me/" -b cookies.txt
```

## Reservar plaza

```
curl -X POST "http://127.0.0.1:8000/api/v1/convocatorias/1/reserve/" \
    -b cookies.txt
```

## Favorito

```
curl -X POST "http://127.0.0.1:8000/api/v1/courses/1/favorite/" \
    -b cookies.txt
```

---

# 18. Checklist global

## Instalación

- [x] `djangorestframework`.
- [x] `django-filter`.

- [x] `drf-spectacular`.
- [x] `django-cors-headers`.
- [x] `requirements.txt` actualizado.

## Configuración

- [x] `THIRD_PARTY_APPS` con las 4 apps.
- [x] `LOCAL_APPS` con `api`.
- [x] `CorsMiddleware` al principio.
- [x] `REST_FRAMEWORK` con permisos, paginación, filtros y schema.
- [x] `SPECTACULAR_SETTINGS`.
- [x] `CORS_ALLOWED_ORIGINS` para React.
- [x] `CSRF_TRUSTED_ORIGINS`.

## Estructura de la app `api`

- [x] `permissions.py`.
- [x] `serializers/` (10 archivos).
- [x] `filters/` (4 archivos).
- [x] `views/` (12 archivos).
- [x] `urls.py`.
- [x] `tests/` (8 archivos).

## Serializers

- [x] `UserPublicSerializer`.
- [x] `AcademySerializer`, `TeacherSerializer`.
- [x] `CategorySerializer`, `CourseSerializer`, `ConvocatoriaSerializer`, `CourseImageSerializer`, `CourseFavoriteSerializer`.
- [x] `EnrollmentSerializer`, `WaitlistEntrySerializer`.
- [x] `CourseReviewSerializer`, `AcademyReviewSerializer`.
- [x] `DiscountSerializer`, `PaymentSerializer`, `ScholarshipSerializer`.
- [x] `NotificationSerializer`.
- [x] `EvaluationSerializer`.
- [x] `CompetenceSerializer`, `TalentPassportSerializer`, `PassportEntrySerializer`.
- [x] `LevelTestSerializer`, `LevelTestAttemptSerializer`.

## Permisos

- [x] `IsAcademyOwnerOrReadOnly`.
- [x] `IsCourseAcademyOwnerOrReadOnly`.
- [x] `IsTeacherAcademyOwnerOrReadOnly`.

- [x] `IsConvocatoriaCourseOwnerOrReadOnly`.
- [x] `IsStudentOwnerOrReadOnly`.
- [x] `IsStaffOrReadOnly`.

## Filtros

- [x] `CourseFilter` con `min_price` / `max_price`.
- [x] `AcademyFilter`.
- [x] `EnrollmentFilter`.
- [x] `PaymentFilter`.

## Autenticación

- [x] `POST /api/v1/auth/register/`.
- [x] `POST /api/v1/auth/login/`.
- [x] `POST /api/v1/auth/logout/`.
- [x] `GET /api/v1/auth/me/`.
- [x] `PATCH /api/v1/auth/me/`.
- [x] `POST /api/v1/auth/change-password/`.

## ViewSets

- [x] `UserViewSet`.
- [x] `AcademyViewSet`.
- [x] `TeacherViewSet`.
- [x] `CourseViewSet`.
- [x] `CategoryViewSet`.
- [x] `CourseFavoriteViewSet`.
- [x] `EnrollmentViewSet`.
- [x] `ConvocatoriaReserveViewSet`.
- [x] `CourseReviewViewSet`.
- [x] `AcademyReviewViewSet`.
- [x] `MyReviewViewSet`.
- [x] `DiscountViewSet`.
- [x] `PaymentViewSet`.
- [x] `ScholarshipViewSet`.
- [x] `NotificationViewSet`.
- [x] `EvaluationViewSet`.
- [x] `CompetenceViewSet`.
- [x] `TalentPassportViewSet`.
- [x] `PassportEntryViewSet`.
- [x] `LevelTestViewSet`.

## Acciones personalizadas

- [x] `mine` (academies, courses, enrollments, payments, evaluations).
- [x] `teachers`, `courses` (academy).
- [x] `favorite` (academy, course).
- [x] `similar` (course).
- [x] `approve`, `enroll` (enrollment).
- [x] `reserve` (convocatoria).
- [x] `read`, `read_all` (notifications).
- [x] `attempt`, `attempts_mine` (level_tests).
- [x] `approve`, `reject` (scholarship).

## Documentación

- [x] `/api/schema/`.
- [x] `/api/docs/`.
- [x] `/api/redoc/`.
- [x] `SERVE_INCLUDE_SCHEMA = False`.
- [x] `@extend_schema` en acciones.

## Tests

- [x] Auth.
- [x] Courses.
- [x] Academies.
- [x] Enrollments.
- [x] Reviews.
- [x] Favorites.
- [x] Talent.
- [x] Stats.

---

# 19. Registro de sesión

```
PROYECTO: Academy Hub
SESIÓN: 08
FASE: API y lógica de negocio
TEMA: Django REST Framework

OBJETIVO:
Crear una API REST completa de Academy Hub con DRF, filtros,
```

```
permisos, documentación y tests.

TEORÍA:
- REST, HTTP, JSON, códigos de estado
- Serializer, ModelSerializer, nested serializers
- APIView, Generic Views, ViewSet, ModelViewSet
- Router
- Autenticación, permisos, permisos de objeto
- Filtering, Search, Ordering, Pagination
- OpenAPI, Swagger, ReDoc
- CORS

IMPLEMENTACIÓN:
- App api con estructura modular
- Serializers por app
- Filtros por app
- ViewSets por app
- Autenticación por sesión
- Acciones personalizadas
- Routers y URLs
- Documentación Spectacular
- CORS para React

ENDPOINTS:
- /api/v1/auth/register/
- /api/v1/auth/login/
- /api/v1/auth/logout/
- /api/v1/auth/me/
- /api/v1/auth/change-password/
- /api/v1/users/
- /api/v1/academies/ (CRUD + mine + teachers + courses + favorite)
- /api/v1/teachers/
- /api/v1/courses/ (CRUD + mine + similar + favorite)
- /api/v1/categories/
- /api/v1/course-favorites/
- /api/v1/enrollments/ (CRUD + mine + academy + approve + enroll)
- /api/v1/convocatorias/{pk}/reserve/
- /api/v1/course-reviews/
- /api/v1/academy-reviews/
- /api/v1/my-reviews/
- /api/v1/discounts/
- /api/v1/payments/ (CRUD + mine)
- /api/v1/scholarships/ (CRUD + approve + reject)
```

```
- /api/v1/notifications/ (read + read_all)
- /api/v1/evaluations/ (CRUD + mine + academy)
- /api/v1/competences/
- /api/v1/passport/ (mine)
- /api/v1/passport-entries/
- /api/v1/level-tests/ (attempt + attempts_mine)
- /api/v1/stats/
- /api/schema/
- /api/docs/
- /api/redoc/

ARCHIVOS CREADOS:
api/__init__.py
api/apps.py
api/permissions.py
api/urls.py
api/serializers/*.py (10)
api/filters/*.py (4)
api/views/*.py (12)
api/tests/*.py (8)

ARCHIVOS MODIFICADOS:
config/settings/base.py
config/urls.py
requirements.txt

TESTS:
- auth: registro, login, me
- courses: listado, detalle, crear, permisos, búsqueda, filtros
- academies: listado, detalle, mine, búsqueda
- enrollments: crear, mine, reserve
- reviews: listado, crear, validación rating
- favorites: crear, propietario no puede, anónimo no puede
- talent: mine
- stats: público

COMANDOS:
pip install djangorestframework django-filter drf-spectacular django-cors-headers
pip freeze > requirements.txt
python manage.py check
python manage.py test
python manage.py runserver
```

```
COMMIT:
feat: add REST API with DRF, filters, permissions and documentation

ESTADO:
[ ] Pendiente
[ ] En progreso
[ ] Completada

PRÓXIMA SESIÓN:
Ofertas, contraofertas e intercambios (Sesión 9).
```

---

# 20. Commit recomendado

```
git status
git add .
git commit -m "feat: add REST API with DRF, filters, permissions and documentation"
```

---

# Resultado final

Antes:

```
Browser → Django → Templates → HTML
```

Ahora:

```
                    ┌→ Web Django → Templates → HTML
Cliente (web/React)─┤
                    └→ API REST → DRF → JSON
                                  ↓
                            Services / ORM
                                  ↓
                            PostgreSQL
```

Ambas vías comparten:

- Modelos.
- Servicios (`reserve_place`, `archive_course`, `verify_academy`).
- Reglas de negocio.

- Base de datos.

Un frontend React + Vite puede consumir **todo** lo que hoy se ve en los templates (cursos, academias, profesores, convocatorias, inscripciones, pagos, notificaciones, evaluaciones, pasaporte, pruebas de nivel, favoritos, stats), con autenticación por sesión y CORS configurado.

La API está documentada automáticamente en `/api/docs/` y `/api/redoc/`.

Los tests cubren los permisos, la búsqueda, los filtros, los favoritos y las acciones personalizadas.